

Mouse Events & the Event Object

mouseover, mouseout, mousemove — and reading data off the event object itself.

HTML

CSS

Mouse Events

Event Object

1 Complete HTML + CSS Code

```
index.html

<!doctype html>
<html lang="en">
  <head>
    <meta charset="UTF-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <title>Document</title>
    <style>
      * {
        margin: 0;
        padding: 0;
        font-family: "Gilroy";
        box-sizing: border-box;
      }

      html,
      body {
        width: 100%;
        height: 100%;
      }

      body {
        overflow: hidden;
        position: relative;
      }

      #abcd {
        position: absolute;
        transition: all 0.15s cubic-bezier(0.19, 1, 0.22, 1);
        width: 400px;
        height: 400px;
        background-color: red;
      }
    </style>
  </head>

  <body>
    <div id="abcd"></div>

    <script src="mouseover.js"></script>
  </body>
</html>
```

2 Complete JavaScript Code

```
mouseover.js

//--- mouseover and mouseout
// let abcd = document.querySelector("#abcd");

// abcd.addEventListener("mouseover", function () {
//   abcd.style.backgroundColor = "yellow";
// });

// abcd.addEventListener("mouseout", function () {
//   abcd.style.backgroundColor = "red";
// });

//--mousemove
// let abcd = document.querySelector("#abcd");

// window.addEventListener("mousemove", function (details) {
//   abcd.style.top = details.clientY + "px";
//   abcd.style.left = details.clientX + "px";
//   console.log(details.clientX, details.clientY);
// });

// event object
// event target
// event type
// preventDefault()
```

NOTE

All the blocks above are commented out in the source — they're the three experiments this file walks through one at a time below.

3 mouseover Event

Fires when the mouse pointer moves **onto an element**.

```
abcd.addEventListener("mouseover", function () {
  abcd.style.backgroundColor = "yellow";
});
```

When mouse enters the red box:

red



yellow

4 mouseout Event

Fires when the mouse pointer moves **away from an element**.

```
abcd.addEventListener("mouseout", function () {
  abcd.style.backgroundColor = "red";
});
```

So together:

```
abcd.addEventListener("mouseover", function () {  
  abcd.style.backgroundColor = "yellow";  
});  
  
abcd.addEventListener("mouseout", function () {  
  abcd.style.backgroundColor = "red";  
});
```

Mouse enters → yellow

Mouse leaves → red

5 mousemove Event

Fires continuously while the mouse moves.

```
window.addEventListener("mousemove", function (details) {  
  // code  
});
```

Because it is attached to `window`, it detects mouse movement across the page.

6 clientX and clientY

The event object gives the mouse's position inside the browser viewport.

```
details.clientX  
details.clientY
```

clientX → horizontal position

clientY → vertical position

Example:

```
window.addEventListener("mousemove", function (details) {  
  console.log(details.clientX, details.clientY);  
});
```

7 Moving the Element With the Mouse

```
window.addEventListener("mousemove", function (details) {  
  abcd.style.top = details.clientY + "px";  
  abcd.style.left = details.clientX + "px";  
});
```

The mouse coordinates are assigned to the element's position.

Mouse X	→ left
Mouse Y	→ top

The `"px"` is required because CSS expects a length unit.

```
details.clientX + "px"
details.clientY + "px"
```

8 Why position: absolute?

```
#abcd {
  position: absolute;
}
```

This allows JavaScript to control:

top	– vertical offset
left	– horizontal offset

For example:

```
abcd.style.top = "200px";
abcd.style.left = "300px";
```

9 Event Object

The parameter:

```
function (details)
```

receives the **event object**. You can call it anything:

```
function (event)
function (e)
function (details)
```

It contains information about the event. Example:

```
window.addEventListener("mousemove", function (details) {
  console.log(details);
});
```

10 Event Target

`event.target` tells you **which element triggered the event**.

```
element.addEventListener("click", function (event) {  
  console.log(event.target);  
});
```

If you click a button, `event.target` refers to that button.

11 Event Type

`event.type` tells you **what event occurred**.

```
element.addEventListener("click", function (event) {  
  console.log(event.type);  
});
```

Output:

click

Other examples:

mouseover · mouseout · mousemove · click · keydown · submit · input · change

12 preventDefault()

Stops the browser's default action.

```
event.preventDefault();
```

Commonly used with forms:

```
form.addEventListener("submit", function (event) {  
  event.preventDefault();  
});
```

Stops the page from refreshing/navigating after form submission.

Important event object properties

Property	Purpose
<code>event.target</code>	Element that triggered the event
<code>event.type</code>	Type of event
<code>event.clientX</code>	Mouse X position
<code>event.clientY</code>	Mouse Y position
<code>event.key</code>	Key pressed
<code>event.data</code>	Data entered in an input event

Property	Purpose
<code>event.target.value</code>	Current input value
<code>event.target.files</code>	Selected files
<code>event.preventDefault()</code>	Stops default browser action

14 Quick Revision

Mouse events

```
mouseover  
mouseout  
mousemove
```

Mouse coordinates

```
event.clientX  
event.clientY
```

Event information

```
event.target  
event.type
```

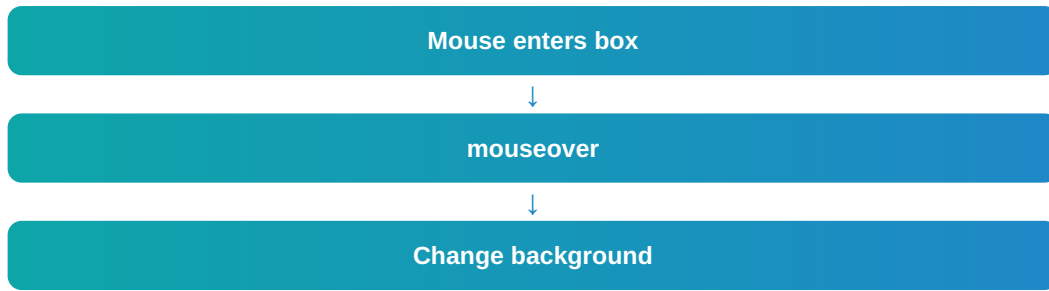
Stop default action

```
event.preventDefault();
```

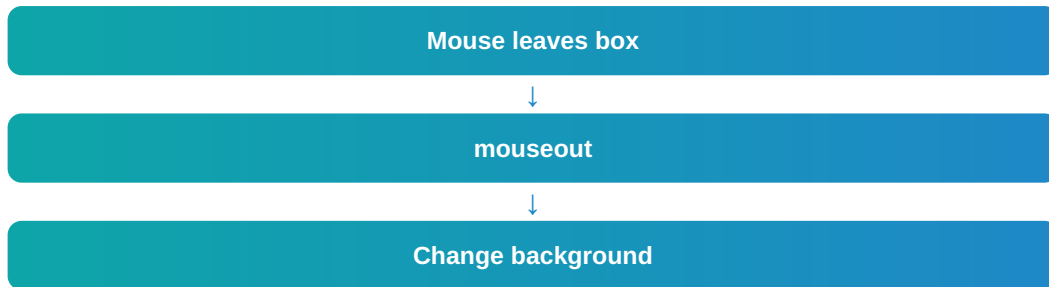
General pattern

```
element.addEventListener("event", function (event) {  
    // use event information  
});
```

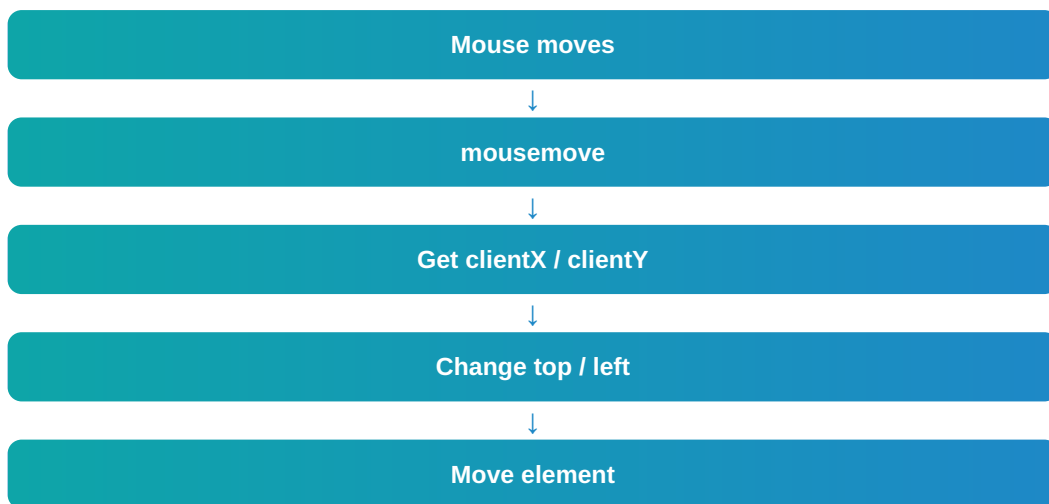
HOVER INTERACTION



LEAVE INTERACTION



TRACKING INTERACTION



🎯 CORE CONCEPT

The event object carries information about what happened, and properties like `target`, `type`, `clientX`, and `clientY` let you use that information in JavaScript.

✨ Mouse events + the event object — the foundation for almost every interactive DOM feature.